Business 41204

Machine Learning

Introduction and Supervised Learning Basics

Outline:

1. Introduction
2. Supervised Learning Setup
3. Validation
4. Tuning

Course Goals

Understand key ML concepts and paradigms

- Surface understanding of key modeling approaches/algorithms
- Stylized but realistic scenarios/applications
  - Will require some coding – LLMs are your friend
- NOT an AI course – though related

Course is NOT meant to make you an AI/ML engineer

- Hopefully does give some skills in this direction
- Helps you meaningfully interact with AI/ML teams

1. Introduction

What is Machine Learning?

Subfield of AI that uses algorithms trained on data to perform desired tasks

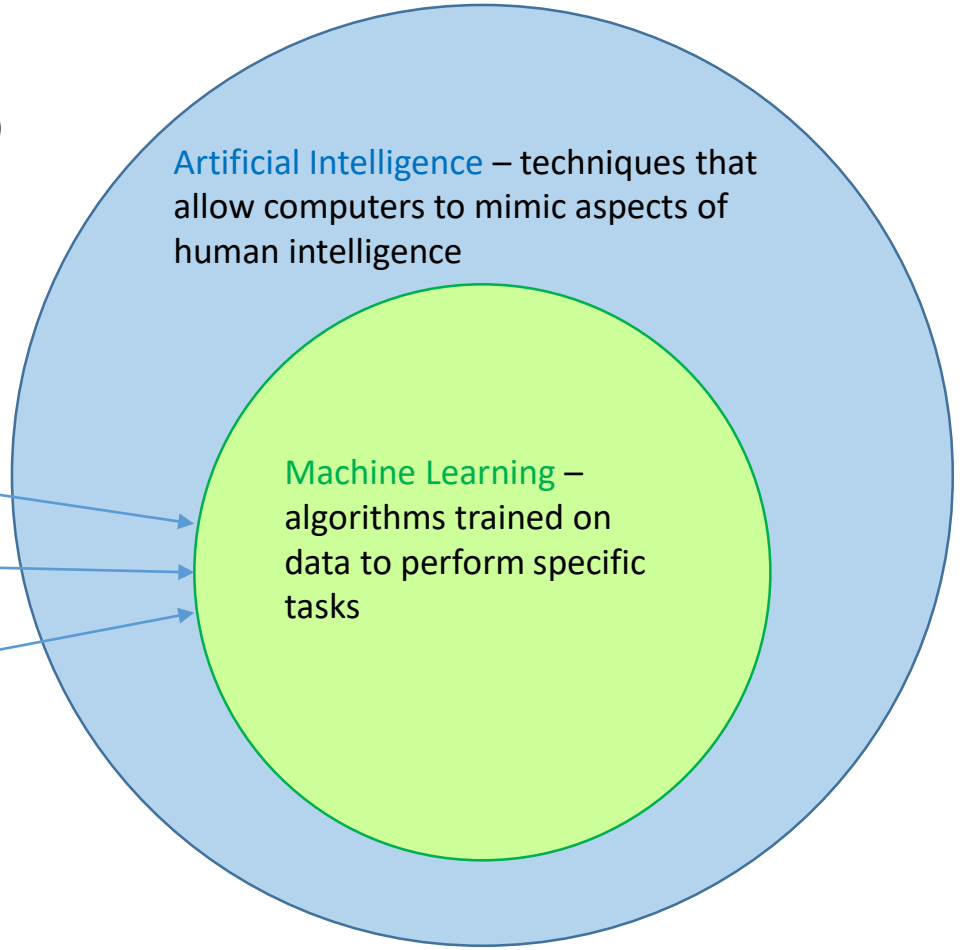
AI: create “autonomous” systems
ML: use machines to learn patterns from data (input into many AI systems)

Prominent Types of Machine Learning

Supervised Learning:
Learn with *labeled data*

Unsupervised Learning:
Extract patterns from *unlabeled data*

Reinforcement Learning:
Learn from reward maximization by exploration (creating *new data*)



What is Machine Learning?

Subfield of AI that uses algorithms trained on data to perform desired tasks

ML tends to be outcome oriented:

Does the algorithm do what we want?

- Focus on evaluation in task we want done
- May care less about understanding all the details about why
  - i.e. many ML algorithms are "*black box*"

Source: https://xkcd.com/1838/ (accessed 7/15/25)

Supervised Learning

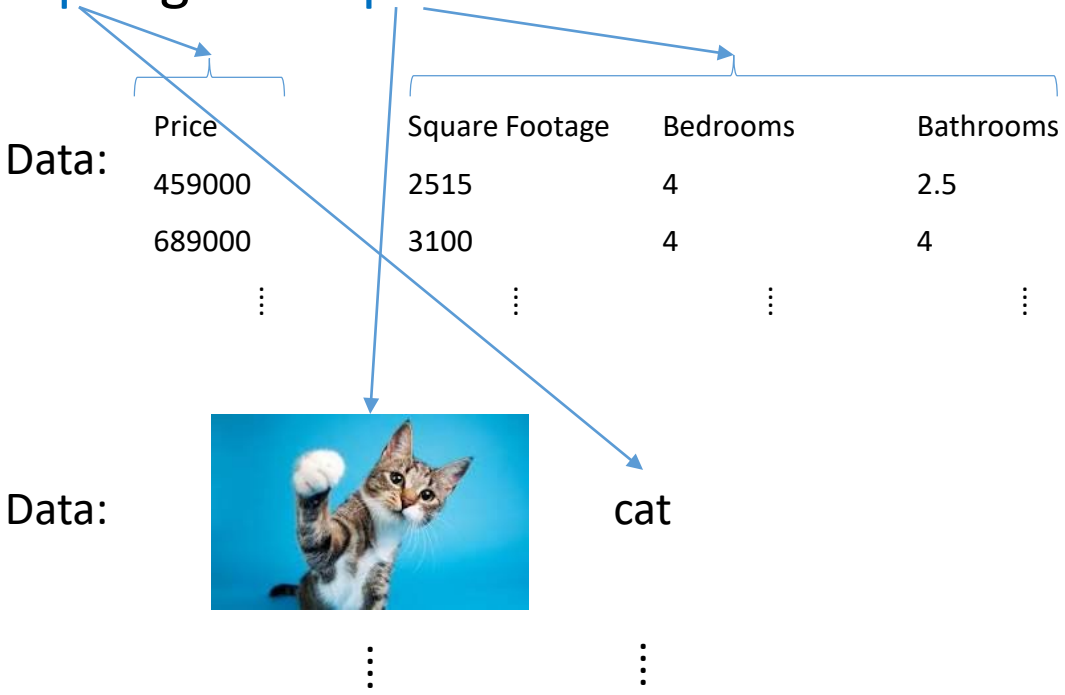
Data come in *input (X)/output (Y)* pairs = *labeled* data

- Lots of jargon that means essentially the same thing
  - output = label, outcome, dependent variable, …
  - input = features, predictors, explanatory variables, …

Goal: learn to *predict* output given input

Examples:

- Revenue forecasting
- House price forecasting
- Demand prediction
- Customer churn/retention
- Image classification
- Asset return forecasting
- Volatility forecasting



Unsupervised Learning

Data is treated as *examples* – no specific input or output

- aka unlabeled data

Goal: learn *structure* that *unifies* examples
(intentionally vague)

Examples:

- Market segmentation
- Recommender systems
- Dimension reduction
- Missing data imputation
- Anomaly detection
- Generative modeling

  e.g., text/image generation

Data:

Customer_ID	Age	Annual Spend $	Online Visits 30D	Pref Channel
C001	28	12,450	18	Mobile App
C002	45	3,200	3	In-Store
C003	33	22,890	25	Website

⋮

Data:

⋮

Reinforcement Learning

Data generated by *interacting* with *environment*

- Data is created within the learning process

Examples:

- Gaming (e.g. AlphaGo)
- Self-driving cars
- Autonomous robots
- Industrial automation
- Algorithmic pricing
- Inventory/supply chain management

Other

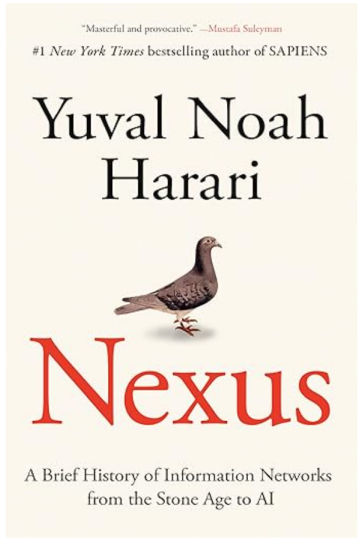
Lots of other learning paradigms – basically just combinations/variants of the three main ones

E.g.

- Semi-supervised learning: small labeled and large unlabeled data
- Self-supervised learning: “mask” elements of data to use as labels
  - Part of LLM/foundation model training
- Offline reinforcement learning: fixed dataset including actions and rewards (somewhere between RL and SL)
- Transfer learning: use model trained in one environment as starting point for model in different environment
- …

Example

Books › Computers & Technology › Computer Science › AI & Machine Learning › Intelligence & Semantics



Read sample | ▶ Audible sample

Nexus: A Brief History of I to AI Hardcover – September 10

by Yuval Noah Harari (Author)

4.6 ★★★★★ (7,127) 4.2 on Goodreads

#1 Best Seller in History of Civilization & Culture

#1 *NEW YORK TIMES* BESTSELLER • From t networks have made, and unmade, our wo

"Strikingly original . . . A historian whose zeitgeist perfectly."—*The Economist*

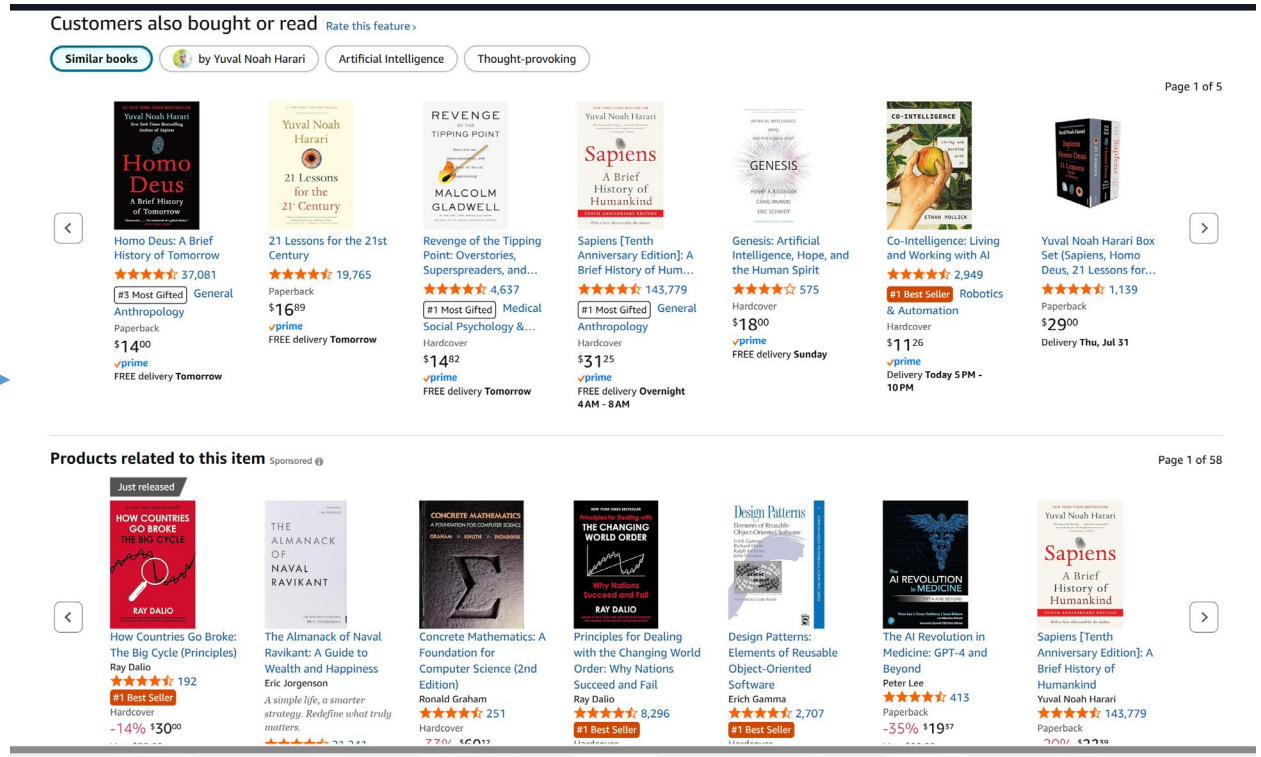
"This deeply important book comes at a c content production. . . . Masterful and pro

For the last 100,000 years, we Sapiens have conquests, we now find ourselves in an exist abounds. And we are rushing headlong into all that we have accomplished, why are we s

Read more

Report an issue with this product or sel

Print length Language



Example

AI· CONSTRUCTION

How a bulldozer, crane, and excavator rental company is using AI to save 3,000 hours per week

BY **SAGE LAZZARO**
CONTRIBUTING WRITER

July 23, 2025 at 10:00 AM EDT

Throughout the recent years of rapid technological innovation, one of the world's largest industries has lagged behind: construction.

Despite moving $10 trillion every year, the sector has averaged just 1% productivity growth over the past two decades compared to 3.6% for manufacturing and 2.8% for the total world economy, according to a McKinsey report. Construction also ranked last for perceived innovation in a survey of 600 U.S. workers, who deemed the field to be "the least technologically competent" out of 10 industries. This lag comes with serious costs: Research from the Saïd Business School at Oxford University found that over 90% of the world's infrastructure projects are late or over budget. And in the U.S. alone, $177 billion is wasted annually due to inefficiencies, according to a survey of 600 construction leaders.

To tackle a small piece of this, BigRentz—a California-based company that since 2012 has matched contractors with rental yards for heavy equipment like forklifts, backhoes, and excavators across the U.S.—reinvented its business from one still operating via phone calls to one running completely on AI that it built internally from the ground up. The models are old-school machine learning, showing there's still value in earlier AI techniques other than large language models. Now the company is launching a stand-alone software platform for large contractors, which is powered by the same AI system but allows customers to run smarter procurement on their existing lists of suppliers.

https://fortune.com/2025/07/23/ai-construction-efficiency-contractors/
(Accessed 7/25/25)

Example

SEARCH FORTUNE MY ACCOUNT

HOME NEWS FORTUNE 500 TECH FINANCE LEADERSHIP LIFESTYLE MULTIMEDIA

AI · AIRLINE INDUSTRY

Delta moves toward eliminating set prices in favor of AI that determines how much you personally will pay for a ticket

BY IRINA IVANOVA
DEPUTY US NEWS EDITOR

July 16, 2025 at 6:48 AM EDT
Updated July 16, 2025 at 4:12 PM EDT

Each passenger has their specific price, and AI is helping Delta find it.
GETTY IMAGES

https://fortune.com/2025/07/16/delta-moves-toward-eliminating-set-prices-in-favor-of-ai-that-determines-how-much-you-personally-will-pay-for-a-ticket/
(Accessed 7/18/25)

Example

ChatGPT 4o

Share

How is an LLM related to supervised, unsupervised, and reinforcement learning?

Large Language Models (LLMs) like GPT-4 are primarily trained using **supervised learning**, but their development and application also involve elements of **unsupervised learning** and **reinforcement learning**. Here's how each learning paradigm relates to LLMs:

---

1. **Supervised Learning (Core Training Stage)**

- **How it's used**: LLMs are trained on large corpora of text using **next-token prediction**, which is a form of supervised learning.
- **Example**: Given the input "The capital of France is," the model learns to predict "Paris" based on similar examples in the training data.
- **Why it counts as supervised**: Even though labels aren't explicitly provided by humans, the next word in the text serves as the label. This makes it *self-supervised*, a subset of supervised learning.

---

2. **Unsupervised Learning (Pretraining Data Collection)**

- **How it's related**: The data LLMs learn from is **not labeled** in the traditional sense (e.g., sentiment labels or class categories). Instead, the model learns patterns, grammar, facts, and reasoning from raw text.
- **Why it's relevant**: The process of gathering huge unlabeled text corpora (like books, web pages, etc.) and learning structure from them is inherently unsupervised. The *learning objective* (e.g., next-token

Ask anything

Tools

ChatGPT can make mistakes. Check important info.

2. Supervised Learning Setup

Supervised Learning

Basic goal:

Produce rule to use input X to predict target Y

- X represents observed features such as
  - Customer demographics – age, income, education, …
  - Product characteristics – price, text description, product image, …
- Y represents observed outcome/target/label such as
  - Whether customer purchases product
  - Quantity of product sold in last week
  - Type of animal in an image

Conceptualization

Useful to think about relation between target (Y) and input (X) as

$$\underbrace{Y}_{\text{target}} = \underbrace{f(X)}_{\text{signal}} + \underbrace{\varepsilon}_{\text{noise}}$$

- $f(X)$ is rule that takes input X and produces a prediction for Y: $\hat{Y} = f(X)$
- ε captures the "noise" or irreducible error in predicting Y
  - E.g. imagine X represents the size of a house – not all houses of the same size have the same price – with only size, even the best prediction rule will have error

GOAL: Use data on Y, X to learn a forecast rule $f(X)$ that produces small forecast error

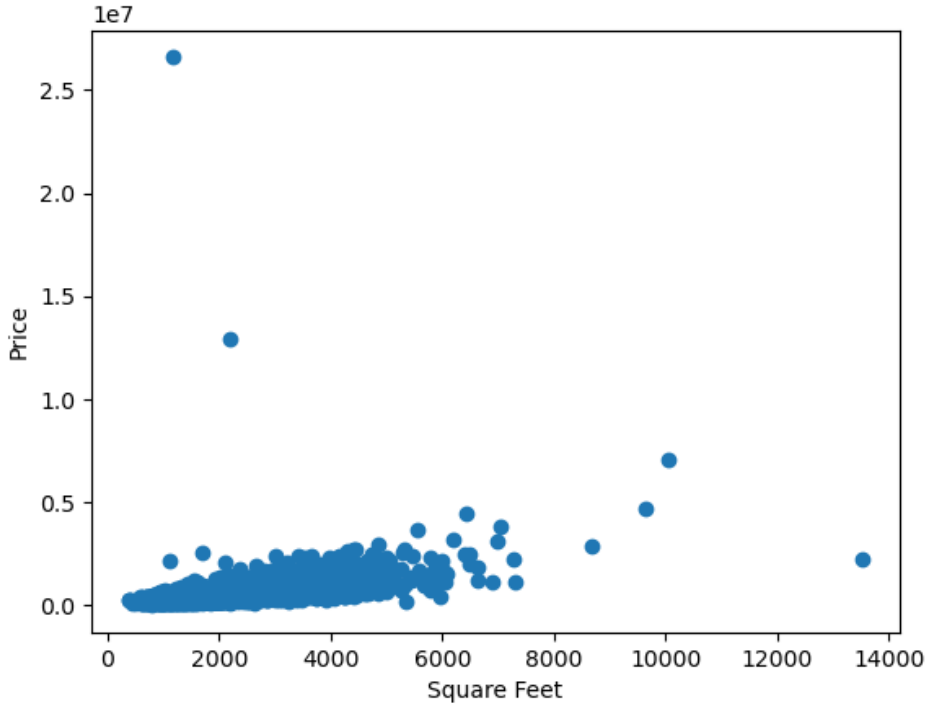
Example: House Price Prediction

Suppose we are interested in predicting sales prices of houses

- Y = Price in USD
- X = easily observed features of a house (e.g. square footage)

Data:

- $n = 4551$ houses that sold in the Seattle, WA area May-July 2014.
  - Going to set aside 1000 observations…
    - Using only 3551 of the available observations
  - What should we think about if we wanted to predict house prices today?
- $p = 18$ raw features



[Notebook for House Price Example]

Loss Function

Need a way to judge what we mean by “small forecast error”.

Use a loss function: $L\big(Y, f(X)\big)$

Common loss functions for “regression”:

- Squared error loss: $L\big(Y, f(X)\big) = \big(Y - f(X)\big)^2$
- Absolute error loss: $L\big(Y, f(X)\big) = |Y - f(X)|$

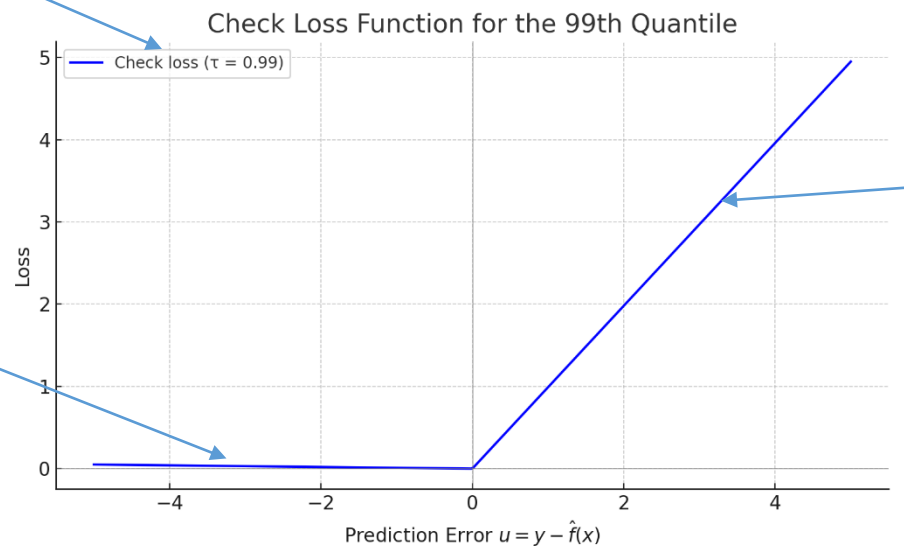
In principle, can use more tailored loss functions.

An Asymmetric Loss Function

Check loss: $L(Y, f(X); \tau) = \big(Y - f(X)\big)\big[\tau - 1\big(Y < f(X)\big)\big]$

- Solution in population is the τ quantile
  - E.g. $\tau = .5$ returns (conditional) median as prediction rule
    - Equivalent to absolute error loss in this case
  - E.g. $\tau = .99$ returns (conditional) 99th percentile as prediction rule

Very low cost for "underpredictions"



High cost for overprediction

When would a loss function like this make sense?

Expected Loss Minimization

With our measure of gauging forecast quality defined, can carefully define our target:

Produce a prediction rule $f(X)$ that provides small *expected* loss when used to forecast an *unseen* outcome Y for a new observation with characteristics X. I.e. find

$$f(X) = \text{argmin}_g \, E[L(Y, g)]$$

- Using notion of a population
- Don't know population expectation, so use a *sample*

Empirical Risk Minimization

Have a sample of $\mathrm{i} = 1, \ldots, n$ observations on

- y_i - outcome variable
- x_i - p predictor/input/feature variables (may be produced by *transformations* of underlying input variables)

ML algorithms typically obtain prediction rules by solving the sample version of expected loss minimization:

$$\hat{f}(X) = \operatorname{argmin} \frac{1}{n} \sum_{i=1}^{n} L\big(y_i, f(x_i)\big)$$

- Computers are pretty good at solving these problems
- Different ML algorithms correspond to different choices for allowable $\hat{f}(X)$

Model Fit

Let $\hat{f}$ be an estimated prediction rule.

Suppose we have $j = 1, \ldots, m$ observations with observed x_j and y_j

- Could be the n observations in the sample used to learn the prediction rule
- **Should be new observations**

Can estimate loss/model fit with sample average loss: $\frac{1}{m}\sum_{j=1}^{m} L\left(y_j, \hat{f}\left(x_j\right)\right)$

E.g.

- Mean squared error ($MSE(\hat{f})$) = $\frac{1}{m}\sum_{j=1}^{m}\left(y_j - \hat{f}\left(x_j\right)\right)^2$
- $RMSE(\hat{f}) = \sqrt{MSE(\hat{f})}$

R^2

Let $\hat{f}$ be an estimated prediction rule and $\hat{f}_0$ be a baseline prediction rule.

Can define $R^2 = 1 - \frac{MSE(\hat{f})}{MSE(\hat{f}_0)}$

- $R^2 \leq 1$
- $R^2 = 1$: predicted values perfectly match observed values on observations used to compute MSE
- $R^2 = 0$: candidate model performance same as baseline model performance
- $R^2 < 0$: candidate model performance worse than baseline model performance
- Typically, baseline model is just sample mean $\left(\hat{f}_0(x) = \bar{Y}\right)$ which predicts the same value no matter the input

"Pseudo R^2"

Let

- $\hat{f}$ be an estimated prediction rule
- $\hat{f}_0$ be a baseline prediction rule
- $L(y, f)$ be a loss function

Can define $"R^2" = 1 - \frac{\frac{1}{m}\sum_{j=1}^{m} L\left(y_j, \hat{f}(x_j)\right)}{\frac{1}{m}\sum_{j=1}^{m} L\left(y_j, \hat{f}_0(x_j)\right)}$

- $"R^2" \leq 1$
- $"R^2" = 1$: predicted values perfectly match observed values on observations used to compute loss
- $"R^2" = 0$: candidate model performance same as baseline model performance
- $"R^2" < 0$: candidate model performance worse than baseline model performance

I.e. can define an R^2 like measure for any loss.

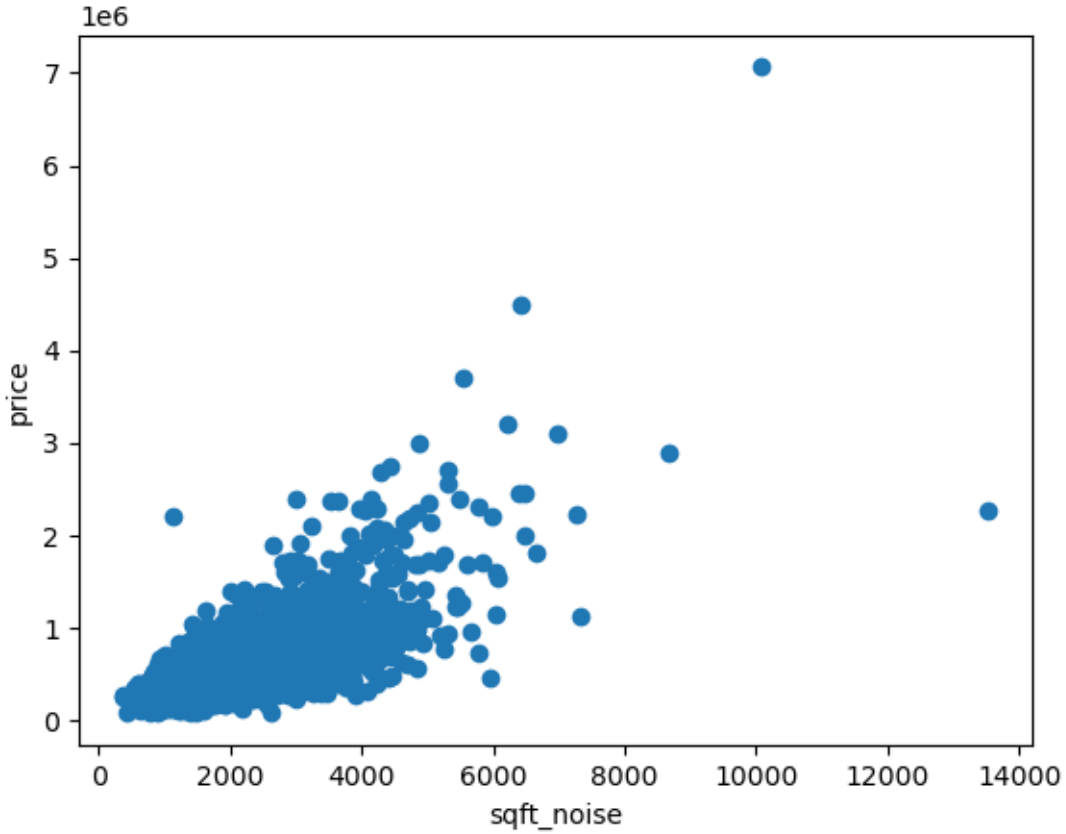
- I'll just refer to all of them as R^2

Example: House Price Prediction

Let's fit some models to predict a house's sales price using square footage (of the living space) as the predictor variable.

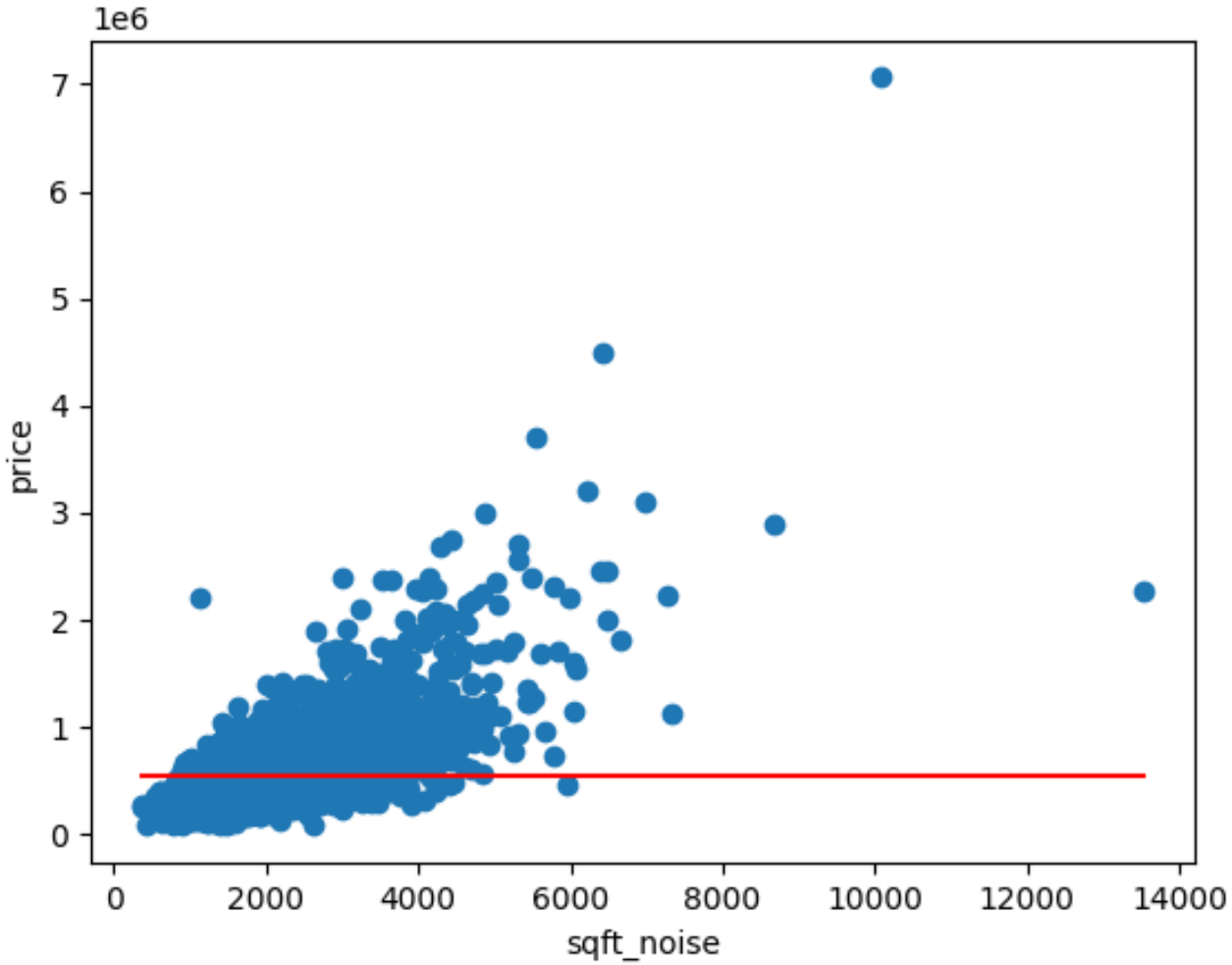
We'll use squared error loss in fitting models.

Here, we've cleaned out some strange seeming observations.



I've added a tiny bit of noise to square footage for the sake of illustration (hence – "sqft_noise")

Baseline Model: Constant/sample mean



Sample mean of outcome: $549,927

MSE of prediction: 135241240351

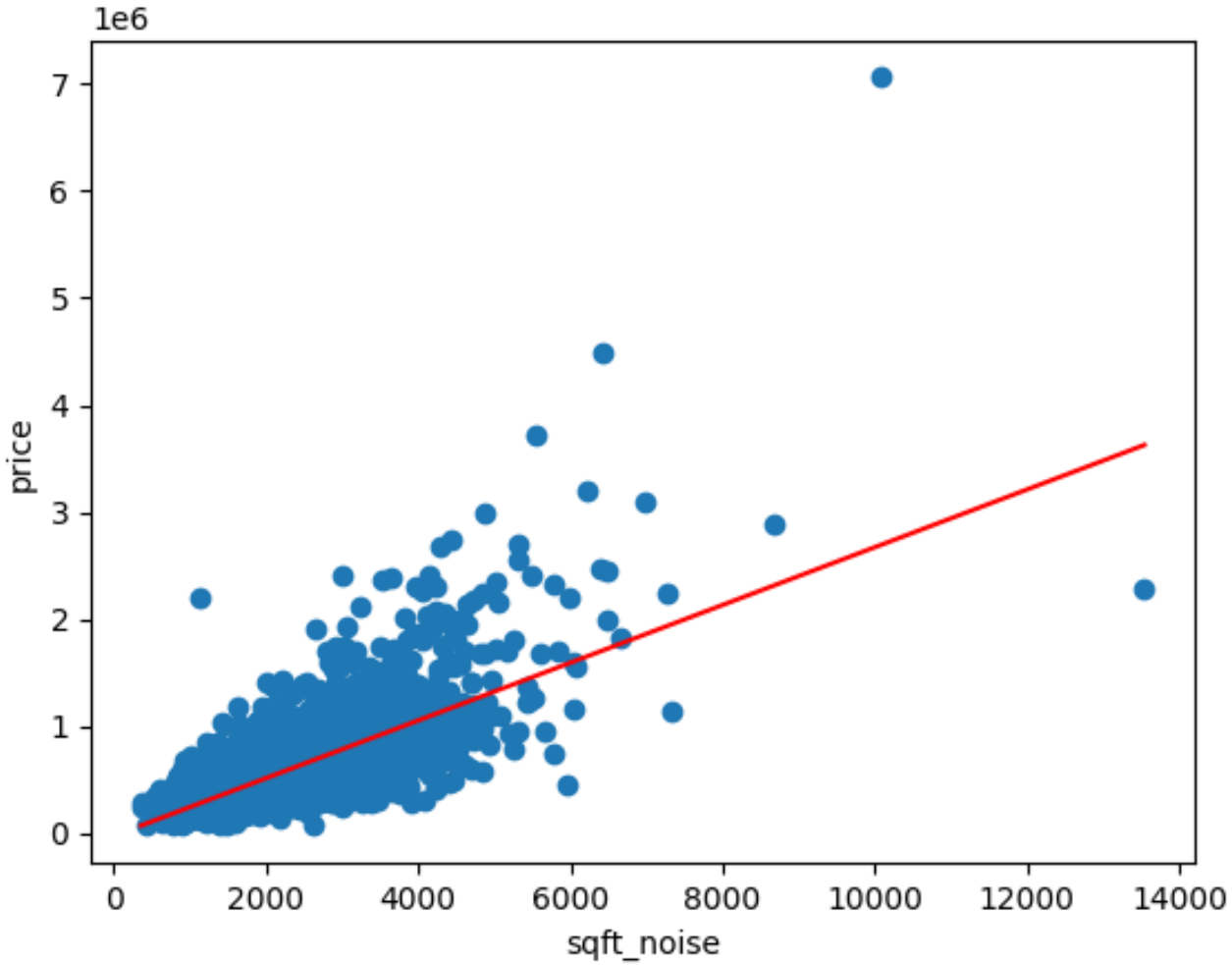
(pretty hard to interpret in isolation)

RMSE: 367,752
MAE: 236,398

Prediction rule ignores size of house.

Can we do better?

Linear Model



Sample mean of outcome: $549,927

MSE of prediction: 68360185216

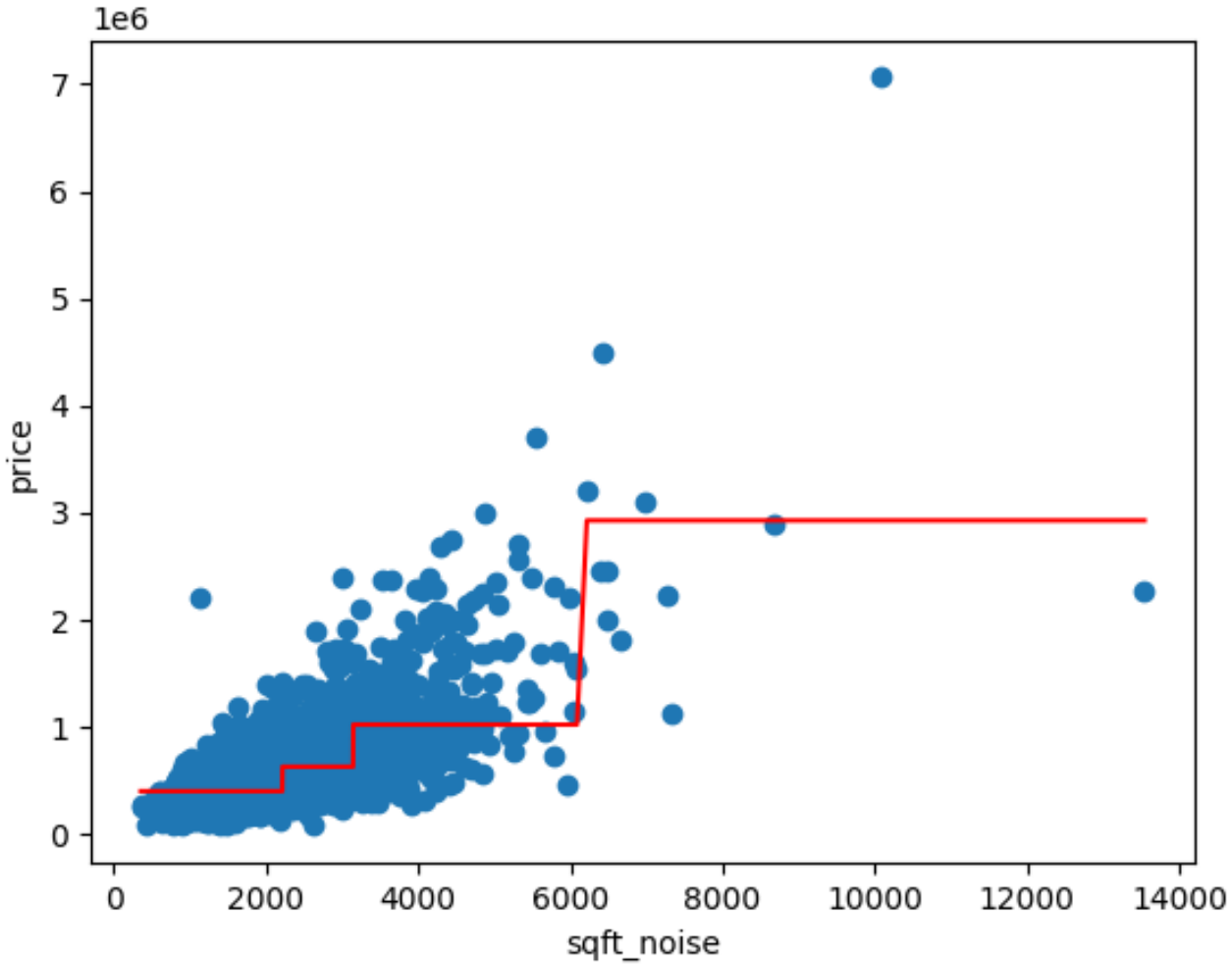
RMSE: 261,458
MAE: 173,035

$R^2 = 0.495$

Recall that using just the sample mean we had

MSE: 135241240351
RMSE: 367,752
MAE: 236,398

Tree with four leaves



Sample mean of outcome: $549,927

MSE of prediction: 72467737010

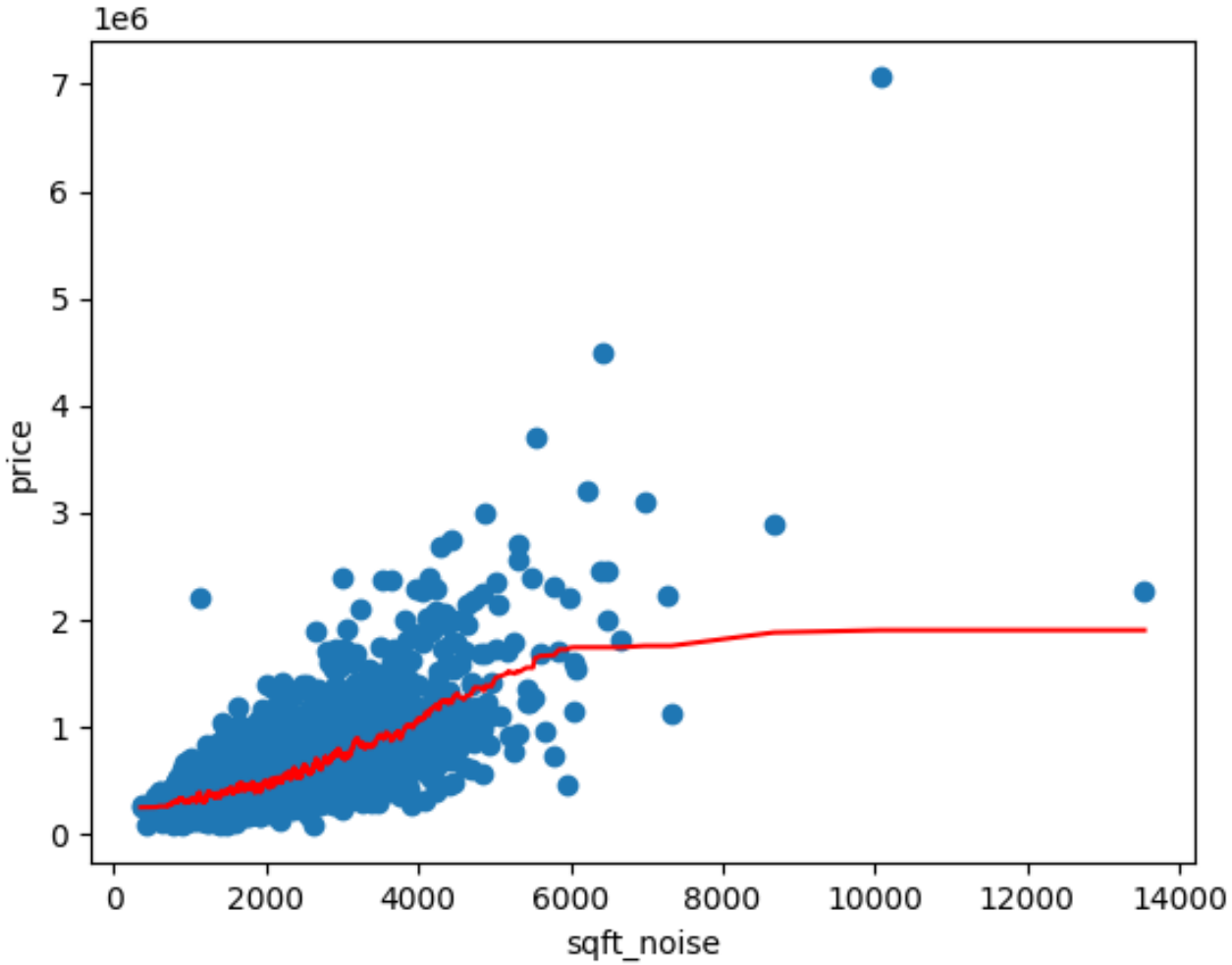
RMSE: 269,198
MAE: 177,425

$R^2 = 0.464$

Recall that using just the sample mean we had

MSE: 135241240351
RMSE: 367,752
MAE: 236,398

KNN with 50 neighbors



Sample mean of outcome: $549,927

MSE of prediction: 67102332575

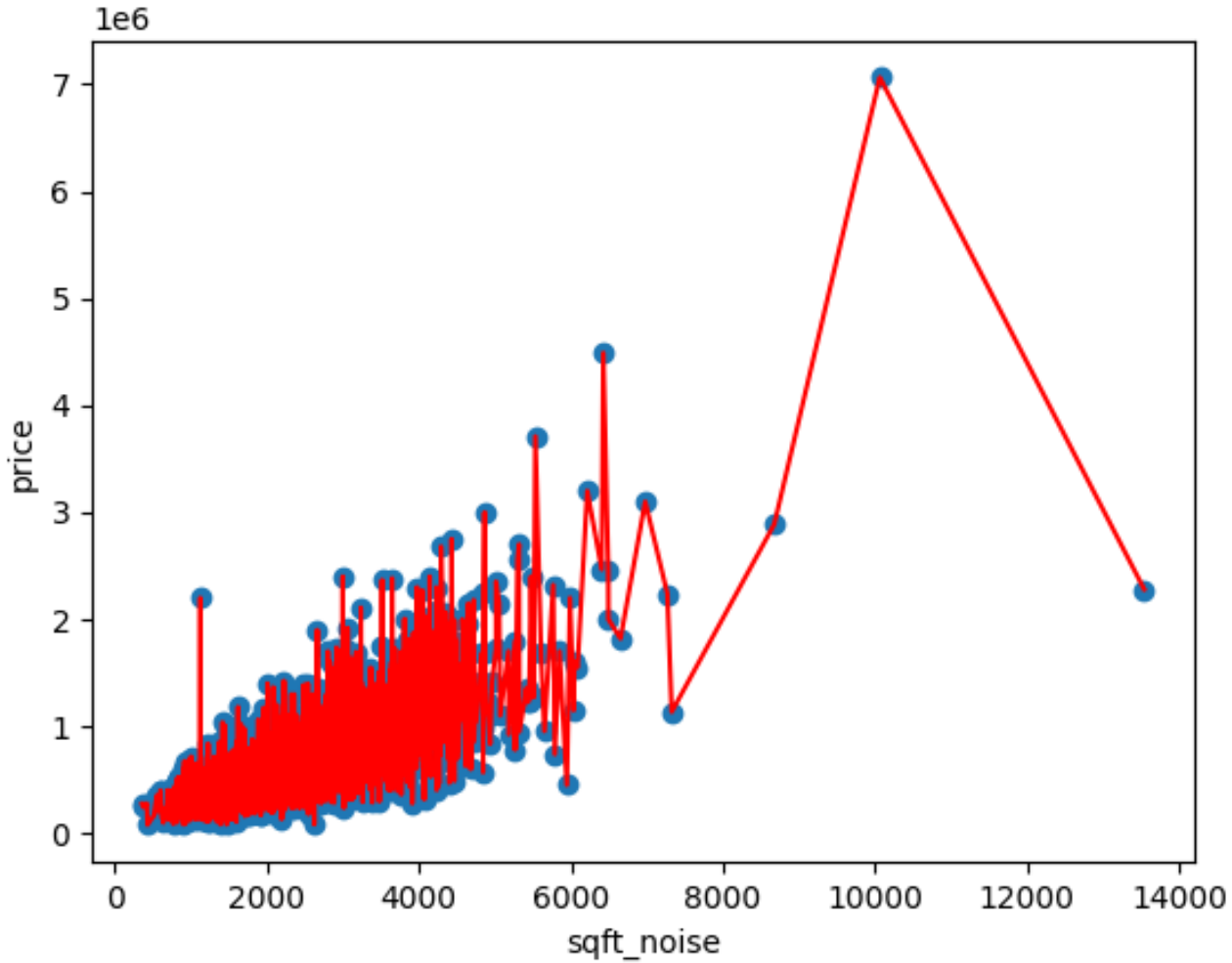
RMSE: 259,041
MAE: 165,685

$R^2 = 0.504$

Recall that using just the sample mean we had

MSE: 135241240351
RMSE: 367,752
MAE: 236,398

Tree with *n* leaves



Sample mean of outcome: $549,927

MSE of prediction: 0

RMSE: 0
MAE: 0

$R^2 = 1$

Recall that using just the sample mean we had

MSE: 135241240351
RMSE: 367,752
MAE: 236,398

Overfitting/Underfitting

Fundamental problem:

- Find model which is not so simple it misses patterns in data
  - avoid underfitting
- Find model which is not so flexible it captures non-generalizable patterns in data
  - avoid overfitting

How do we decide?

With one input and moderate numbers of observations, we can use our eyes and intuition. We want something more general.

3. Validation

Prediction Loss

Remember: Our goal is to build a rule to predict *new* observations ("out-of-sample fit")

- When we build a model, we learn how well that model fits the observations used to build it ("in-sample fit")
- In-sample fit approximates out-of-sample fit when n is large and model is "not complex"
- **Most ML methods are "complex" though**

There are ways to formally define complexity. Rather than worry about this, we're just going to note that most ML methods are complex and that validation exercises work for simple AND complex methods.

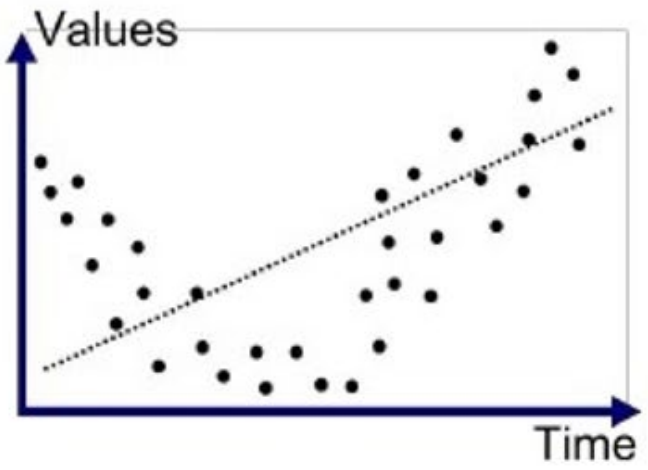
Punchline: We don't judge model quality on in-sample fit but use validation exercises.

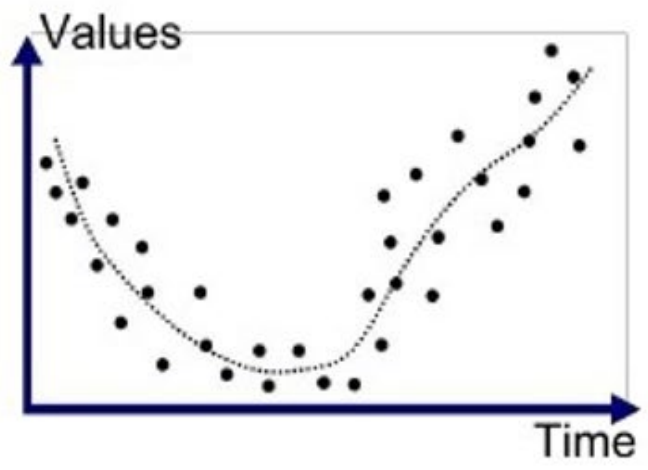
A validation exercise aims to replicate the forecasting environment by splitting data into

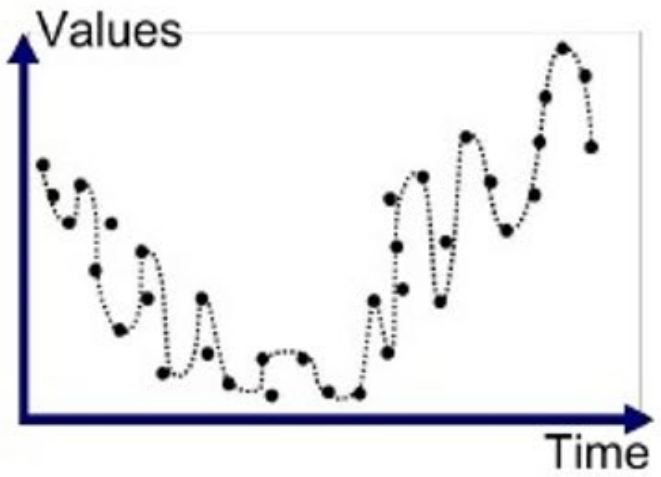
- training data – used to learn prediction rules
- testing/validation/hold-out data – used to test estimated rules on new observations
  - Can use same *or different* loss metrics as for training

Bias/Variance Tradeoff

Can think of "Bias/Variance tradeoff" as shorthand jargon for choosing a rule with the right complexity for the task at hand. Not going to worry about formalizing.







Underfitted

When we build models that are too simple – underfit – we end up with a "biased" prediction rule.

Good Fit/Robust

Good predictive models trade off "bias" and "variance" to land in a sweet spot.

Overfitted

When we build models that are too complex – overfit – we end up with high "variance" prediction rule.

You cannot tell where you are by only looking at the data used to fit the model

⇒ Validation exercise

Train/Validate/Test

Ideally, validation exercises are actually based on splitting the data into three pieces:

- Training data: Data used to fit the candidate prediction rules
  - Want most of the data here
- Validation data: Data used to test/fine tune prediction rules
- Test data: Final data set used to check performance of model before going into production/giving it to client
  - Validation and test data sets typically of similar size

Validation Exercise on House Price Example

How do our simple sales price prediction rules do in a validation exercise?

- Look at prediction performance in a held out sample of 1000 observations

	In-sample			Out-of-sample		
	RMSE	MAE	R^2	RMSE	MAE	R^2
Mean	367752	236398	-	368740	237996	-
Linear	261458	173035	0.495	272404	177069	0.454
Tree 4	269198	177424	0.464	284558	183263	0.404
KNN 50	259041	165685	0.503	278098	173134	0.431
Tree *n*	0	0	1	348463	233181	0.107

Which rule should we use?

What happened to "Tree *n*"?

Cross-Validation

Potential issues with basic validation exercise

- Doesn't use all data for training and testing
- Depends on the specific random split

Cross-validation (CV) aims to alleviate these concerns – essentially by repeating the validation exercise

CV also has issues

- Computationally more burdensome that simple validation exercise
- Have to decide what model to use when you're done
- Using the same data repeatedly
- How to structure with data that is not independent?

K-Fold CV Algorithm:

1. Divide sample into K (approximately) equal size groups
2. For $k = 1, \ldots, K$
    1. Set aside subsample k as validation data
    2. Use remaining subsamples as training data. Call these training data T_k
    3. Use training data T_k to estimate candidate prediction rules, $\hat{f}_k(x)$
    4. Form predictions for observations i in subsample k, $\hat{y}_i = \hat{f}_k(x_i)$
3. Estimate forecast loss as $\frac{1}{n}\sum_{i=1}^{n} L(y_i, \hat{y}_i)$
    - E.g. CV-MSE = $\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$

Cross-validation in House Price Example

Let's see what 5 Fold cross validation looks like in our sales price prediction example

- 5 Fold CV here is based on the training observations (i.e. identical to the **In-sample** data)

	In-sample		**5-Fold CV**		**Out-of-sample**	
	RMSE	R^2	RMSE	R^2	RMSE	R^2
Mean	367752	-	366583	-	368740	-
Linear	261458	0.495	260792	0.494	272404	0.454
Tree 4	269198	0.464	297044	0.340	284558	0.404
KNN 50	259041	0.503	263658	0.485	278098	0.431
Tree *n*	0	1	368160	-0.013	348463	0.107

Cross-validation and evaluation in the hold out sample tell roughly the same story.

4. Tuning

Tuning

ML algorithms involve *tuning parameters*

- Algorithm specific choices that strongly influence performance
  - What variables to use? Transformations? Interactions?
  - How many leaves in a tree?
  - How many neighbors for knn?
  - How deep to make a deep neural network?
  - How much to penalize size of coefficients in penalized methods?
  - How many lags to consider in time series models?
- Need to be specified in real use cases
  - Typically chosen by (computerized) trial and error in validation exercises

Parametric Models

Many common prediction algorithms use parametric prediction rules:

$$\hat{Y} = f(X; \hat{\theta})$$

- $\hat{\theta}$ are parameters estimated from the training data

Examples:

- Linear regression: $\hat{Y} = \hat{\theta}_0 + \hat{\theta}_1 X_1 + \cdots + \hat{\theta}_k X_k$
- Logistic regression (for binary classification):

$$\hat{Y} = \sigma\left(\hat{\theta}_0 + \hat{\theta}_1 X_1 + \cdots + \hat{\theta}_k X_k\right)$$
$$\sigma(u) = \frac{1}{1 + e^{-u}} = \frac{e^u}{1 + e^u}$$

- Deep neural networks

Penalized Estimation

To guard against overfitting with parametric prediction rules, it is common to use penalized estimation.

Choose parameters for prediction rule by solving

$$\hat{\theta} = \text{argmin}_\theta \frac{1}{n}\sum_{i=1}^{n} L\big(y_i, f(x_i, \theta)\big) + \lambda p(\theta)$$

- $p(\theta)$ is a penalty function that imposes a cost to having large coefficients
  - ℓ_1-penalization (lasso): $p(\theta) = \sum_k |\theta_k|$
  - ℓ_2-penalization (ridge): $p(\theta) = \sum_k \theta_k^2$
- λ is the tuning parameter that controls the cost of coefficient size
  - typically chosen by (cross-)validation
  - i.e., trial and error – try many values and see which works best in validation exercise

House Price Prediction: Feature Variables

Outside of price, the data includes features

- `date` – date of sale.
- `sqft_living, sqft_lot, sqft_above, sqft_basement` – square footage variables
  - Note `sqft_living = sqft_above+sqft_basement`
- `bathrooms, bedrooms, floors` – numeric variables with small number of values
- `waterfront, view, condition` – ordered categorical
- `yr_built, yr_renovated` – year built and year of most recent renovation
- `street, city, statezip, country` – categorical/qualitative variables

Which should we use? How should we use them?

Notebook for Tuning Example

Linear Regression in House Price Example

Let's look at some simple linear regression models.

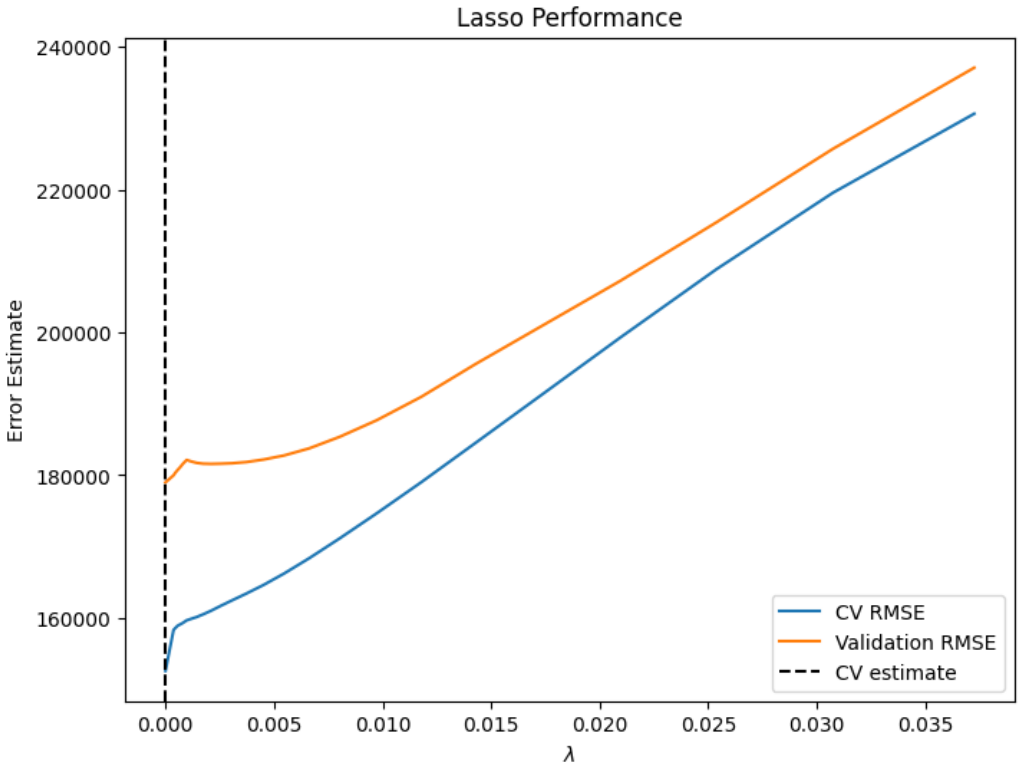
- Note that we are going to use log_price as the outcome variable (and take logs of several of the features). How do we do validation/think about prediction?

Model:

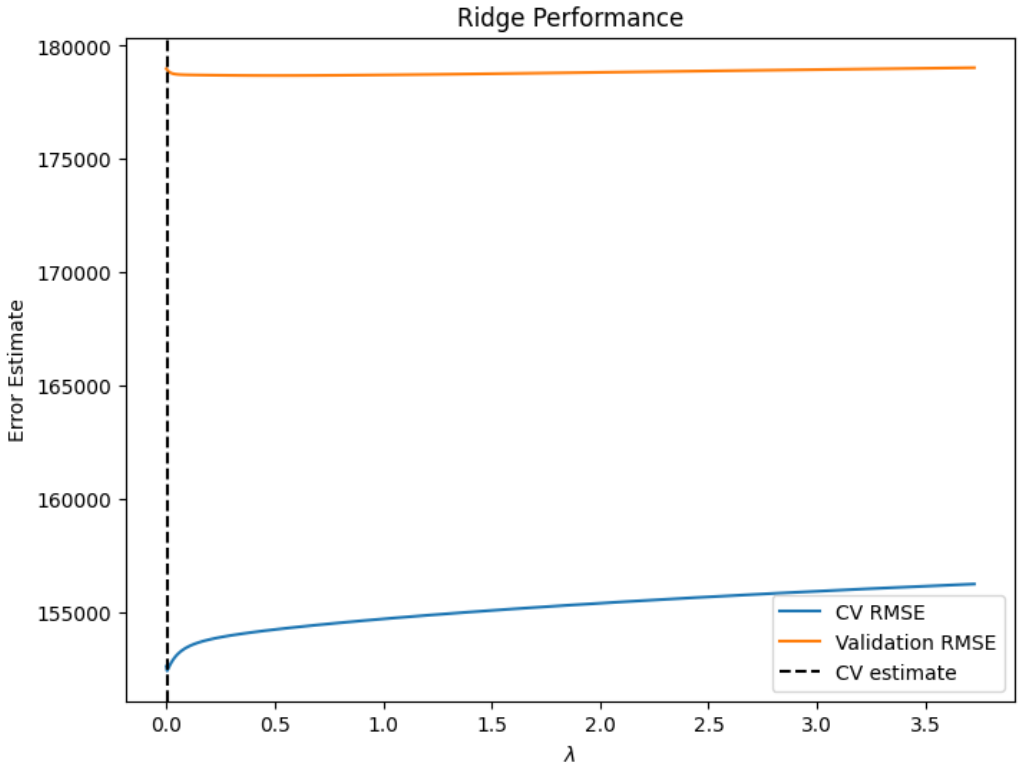
- Use features `sqft_living, sqft_lot, sqft_above, sqft_basement, bathrooms, bedrooms, floors, waterfront, view, condition, yr_built, yr_renovated`
  - Include squares of all "continuous" variables
  - Interact `sqft_living` with `bathrooms` and `bedrooms`
- Use dummy variables for `city, statezip`
  - Simple parametric models often work well with large sets of dummy variables
  - 44 cities and 77 zip codes in data
  - What if we wanted to generalize beyond Seattle area?

Will use penalized regression with two penalty functions (lasso, ridge) and penalty parameter chosen by cross-validation

Tuning



CV R^2 = 0.8280
Validation R^2= 0.7644



CV R^2 = 0.8284
Validation R^2= 0.7645